

Test la Programarea Calculatoarelor - Subiect D

10. ian. 2022

Fiecare problemă va fi rezolvată într-un proiect separat. Fișierele sursă *main.c* de la fiecare proiect în parte se redenumesc *Nume_Prenume_prb1.c* și *Nume_Prenume_prb2.c*, apoi se încarca pe Teams. Nu se punctează cod scris în comentariu.

1p – codul compilează, este încărcat cu nume corect, este formatat cu AStyle (0.5p per problema)

1p – rezultate corecte pe exemplele date în cerință (0.5p per problema)

1p – verificări și teste scrise pentru alte cazuri în comentarii (0.5p per problema)

Problema 1 (3p) - Problema 1

Se consideră un număr $n < 10^9$ și o listă v care conține cel mult 20 de numere prime mai mici decât 1000. Spunem că un număr este urât dacă factorii primi ai lui n se regăsesc doar în lista v .

Exemplu: dacă $v = [2, 3, 5]$, atunci $n = 15(3 * 5)$ este urât, $n = 21(3 * 7)$ nu este urât.

a) Să se citească un numărul n , un număr $k < 1000$ și vectorul de numere prime mai mici decât 1000 și să se salveze.

b) Să se scrie o funcție care primește ca parametru un număr $x < 10^9$ și un vector de cel mult 20 de numere prime mai mici decât 1000 și care determină dacă numărul x este sau nu urât pe baza factorilor primi din vector. Se vor folosi datele citite la punctul a).

Exemplu: $v = [11, 13]$, $n = 11$ este urât, $n = 128$ nu este urât

c) Să se scrie o funcție care primește ca parametru numărul k și un vector de factori primi mai mici decât 1000. Să se determine al k -lea număr urât mai mic decât 1000 pe baza vectorului de factori primi. Dacă acesta nu există, se returnează -1.

Exemplu:

$v = [3, 5]$, $k = 3$, atunci al 3-lea număr urât pentru vectorul v este 9 (primul este 3, al doilea este 5).

$v = [2, 3, 5]$, $k = 66$, al 66-lea număr este 500.

Problema 2 (4p) - Problema 2

Se consideră fișierul *cuvinte.txt* mai mic de 1MB care conține pe prima linie un șir de caractere, iar pe următorul rând un număr necunoscut de cuvinte separate prin spațiu.

a) Să se citească șirul de caractere și cuvântul și să se memoreze folosind alocare dinamică.

b) Să se scrie o funcție recursivă care primește ca argumente două șiruri de caractere s și t și care determină dacă în s apare cel puțin o dată prima literă a șirului t .

Exemplu:

$s = \text{"abc"}$ și $t = \text{"xyz"}$, returnează adevărat

$s = \text{"abc"}$ și $t = \text{"xyz"}$, returnează fals

c) Să se scrie o funcție care primește ca argumente două șiruri de caractere, s și t , și care determină dacă șirul s este format doar din caractere din șirul t .

Exemplu:

$s = \text{"abbcxyz"}$ și $t = \text{"ab"}$ returnează fals.

$s = \text{"abcddeefaaa"}$ și $t = \text{"abcdef"}$ returnează adevărat.

d) Să se scrie o funcție care primește ca argumente șirul de caractere, notat aici t , și cuvintele citite la punctul a). Pentru fiecare cuvânt, funcția trebuie să scrie în fișierul *substring.txt*, pe fiecare linie, cuvântul urmat de substring-urile care conțin doar caractere prezente în șirul de t . Șirurile care se suprapun nu se iau în considerare.

Exemplu:

$t = \text{"abc"}$, cuvânt = "aabcdabbbbcxabc" , în fișier se va scrie: $\text{"aabcdabbbbcx abc abbbbc abc"}$

abc -> de la poziția 1 -> 3

abbbbc -> de la poziția 5 -> 10

abc -> de la poziția 12 -> 14